

```
#define MPT2SAS_MINOR    221
#define HPET_MINOR      228
#define FUSE_MINOR       229
#define KVM_MINOR        232
#define MISC_DYNAMIC_MINOR 255
```

The structure of the *miscdevice* is defined below:

```
struct miscdevice {
    int minor;
    const char *name;
    const struct file_operations *fops;
    struct list_head list;
    struct device *parent;
    struct device *this_device;
    const char *devnode;
};
```

The *minor* used in the structure actually refers to the minor number being registered. It is expected that you choose a unique number while registering the driver (since you are going to use this as the pointer to the device from the driver DD1 or DD2). You can easily do this by looking at the entries hard coded (see the code that 'listed' the values) and by 'cat-ing' the */proc/misc* file (which will also show the name of a particular device registered with the misc driver):

```
linux-elbp:/proc/bus/usb # cat /proc/misc
57 vboxmctcl
58 vboxdrv
229 fuse
59 device-mapper
60 rfskill
130 watchdog
175 appgart
61 network_throughput
62 network_latency
63 cpu_dma_latency
1 psaux
144 mvrnm
228 hpet
231 snapshot
227 ncelog
linux-elbp:/proc/bus/usb #
```

As you would have guessed, the *name* used in the structure of the *miscdevice* indicates the name for the device that you are going to handle (just like the names that appeared while reading the entries from */proc/misc* file). And *fops* is used to indicate the file operations that you want to perform on a particular device. Here's the code (which can be found in *miscdevice.h*) that handles this structure:

```
static const struct file_operations misc_proc_fops = {
    .owner  = THIS_MODULE,
    .open   = misc_seq_open,
```

```
.read  = seq_read,
.llseek = seq_lseek,
.release = seq_release,
};
```

In order to register and unregister, you need to have the following lines in the code:

```
#include <linux/miscdevice.h>
int misc_register(struct miscdevice * misc);
int misc_deregister(struct miscdevice * misc);
```

As mentioned before, you need to assign a unique (i.e., it shouldn't be in use) value to the driver and you can do that in a dynamic way as well by using the following lines of code:

```
static struct miscdevice my_device;

int init_module(void)
{
    int my_min_no;
    my_device.minor = MISC_DYNAMIC_MINOR;
    my_device.name = "DD1";
    my_device.fops = &my_fops;
    my_min_no = misc_register(&my_device);
    if (my_min_no) return my_min_no;
    printk("DD1 was assigned minor %i\n", my_device.minor);
    return 0;
}
```

MISC_DYNAMIC_MINOR is used with the *misc_register* function which will register a miscellaneous device. You may note that if you are using MISC_DYNAMIC_MINOR in the code, then a minor number is generated and used in the minor field of the structure (which I showed you earlier). If you are not using this method, then the *misc_register* function will use the number that you have hardcoded in your source.

Here is code that performs the 'registration':

```
int misc_register(struct miscdevice * misc)
{
    struct miscdevice *c;
    dev_t dev;
    int err = 0;

    INIT_LIST_HEAD(&misc->list);

    mutex_lock(&misc_mtx);
    list_for_each_entry(c, &misc_list, list) {
        if (c->minor == misc->minor) {
            mutex_unlock(&misc_mtx);
            return -EBUSY;
        }
    }

    if (misc->minor == MISC_DYNAMIC_MINOR) {
```